Inteligență Artificială

IA aplicat în Prelucrarea Limbajului Natural

Slides: Alexandru Sorici, conținut parțial adaptat după slide-uri de la Princeton (COMP336)

- Modele de Limbaj - Recapitulare
- Reprezentare numerică a textului
- LLM

Cum se modelează dependența unui cuvânt de altele dintr-un document?

- **Cu gramatici**: regulate, dependente de context, independente de context, independente de context și probabilistice
- **La nivel probabilistic**: se calculează **probabilitatea** unei propoziții înțeleasă ca **secvență de cuvinte**: $p(d) = p(w_1, w_2, \dots w_n)$
  - Un obiectiv conex este modelarea probabilității de apariție a unui cuvânt următor $p(w_i|w_{i-1}, w_{i-2}, \dots, w_1)$
  - Un model care calculează $p(d)$ sau $p(w_i|w_{i-1}, w_{i-2}, \dots, w_1)$ se numește model de limbaj



1940-2000

Focus on rule-based systems, formal grammars

Development of linguistic resources (lexicon, ontologies, grammars...)

1990-2010

Statistical learning theory (SVM, Random Forest), Graphical Probabilistic Models (e.g. LDA, HMM)

Development of annotated datasets

2010-Today

Deep Learning Architecture (Transformer)

Transfer Learning in NLP (word2vec, BERT, CamemBERT, GPT)

More compute, larger (raw) dataset, Open Source Deep Learning Libraries

Cum putem obține o **reprezentare numerică a textului** pentru a putea aplica algoritmi de ML cu rețele neurale pe text?

- Word Embedding
  - Reprezentare a vocabularului care **surprinde contextul cuvintelor**
  - **Asociere semantică** – co-apariție în același context (e.g. albină – miere)
  - **Co-apariție de ordinul I** – relații *sintagmatice* (e.g. miere de albină, miere de mană)
  - **Co-apariție de ordinul II** – relații *paradigmatice / de similaritate* (e.g. albină – bondar)

O primă soluție posibilă: 1-hot Encoding

- Metodă de asociere a unui întreg (în reprezentare **binară**) unui **element discret**. În cazul nostru: **cuvânt** din **vocabularul** folosit
- Exemplu: pentru cuvintele *{ 1=film, 2=hotel, 3=măr, 4=film**e**, 5=artă}* obținem reprezentarea 1-hot ca:

$$\begin{aligned} \textit{film} &= [1,0,0,0,0] \\ \textit{hotel} &= [0,1,0,0,0] \\ \textit{mar} &= [0,0,1,0,0] \\ \textit{filme} &= [0,0,0,1,0] \\ \textit{arta} &= [0,0,0,0,1] \end{aligned}$$

1-hot Encoding – **Probleme**

- **Nu este foarte informativ**: e.g. film și film**e** sunt considerate a fi două cuvinte cu encodări total diferite
- Nu avem o noțiune corectă de **distanță** semantică: e.g. $||\textit{hotel} - \textit{film}|| = ||\textit{hotel} - \textit{arta}||$
- **Polisemia** rămâne o provocare: *film* (conținut audiovizual văzut la televizor) ar avea aceeași reprezentare ca *film* (bandă flexibilă de celuloid, transparentă și subțire)
- În cazul unui vocabular mare, vectorii ar devenii foarte mari și cu reprezentare **rară**

Cum putem **infera** o reprezentare bună pornind de la date (un **corpus de text**)?

Ipoteză a **distribuției cuvintelor**:

- Modelăm **contextul** unui cuvânt pentru a obține o **reprezentare vectorială**
- Ne dorim vectori de dimensiuni mai mici (de ex câteva sute), care pot fi determinați automat

Ipoteză a **distribuției cuvintelor**: Exemplu (Dexonline – definiție film)

- A mers să obțină un **film** 6/9 pentru radiografii dentare.
- **Filmele** vor da sătenilor răspuns la multe întrebări... vor limpezi multe nedumeriri.
- Datorită **filmului**, «O scrisoare pierdută» va străluci pe ecrane pretutindeni. Iar peste alte decenii, **filmul**-spectacol «O scrisoare pierdută» va fi un adevărat document al dragostei și venerației noastre, a celor de azi, pentru marele Caragiale.
- Magicul **film** al copilăriei începea din nou să-i ruleze pe ecranul minții.

Ipoteză a **distribuției cuvintelor**: **Procedură**

1. Definirea unui **context** al unui cuvânt într-un document.
2. **Numărarea co-ocurenței** fiecărui cuvânt din context cu cel țintă
3. Construirea unui vector printr-o **funcție** definită peste contoarele de co-ocurență

$$x_w = f(w, context(w))$$

Contextul unui cuvânt poate fi:

- Cuvintele înconjurătoare (la stânga și dreapta cuvântului țintă în document)
- Toate celelalte cuvinte (afară de cel țintă) din document
- Toate celelalte cuvinte **după o preprocesare sau filtrare (e.g. stop words)**

Cum modelăm contextul cuvântului?

$$x_w = f(w, context(w))$$

- Modelare pe bază de **frecvență de apariției (contorizare)**
- Modelare pe bază de **predicție**

Reprezentare numerică a textului : PMI

Cum modelăm contextul cuvântului? **Pe bază de contorizare**
Exemplu: co-ocurența cuvintelor pentru "dog" în exemplul

The dog barked in the park.
The owner of the dog put him on the leash since he barked.

co-occurence # dog	
barked	++
park	+
owner	+
leash	+

Reprezentare numerică a textului : PMI

Cum modelăm contextul cuvântului? **Pe bază de contorizare** – matrice de co-ocurență

	leash	walk	run	owner	pet	barked	the
dog	3	5	2	5	3	2	8
lion	0	3	2	0	1	0	6
light	0	0	0	0	0	0	5
bark	1	0	0	2	1	0	0
car	0	0	1	3	0	0	3

- Vectorizarea se poate citi pe rânduri
- Limitare: reprezentările depind de **dimensiunea corpusului**; cuvinte mai frecvente în limbă impactează reprezentarea; reprezentări sensibile la schimbări în utilizarea cuvintelor rare

Reprezentare numerică a textului : PMI

Cum modelăm contextul cuvântului? **Pe bază de contorizare – Informație Mutuală**

$$PMI(w_1, w_2) = log \frac{p(w_1, w_2)}{p(w_1) \cdot p(w_2)} = log \frac{\frac{1}{n_{perechi}} \cdot contor((w_1, w_2))}{\frac{1}{n_{cuv}} \cdot contor(w_1) \cdot \frac{1}{n_{cuv}} \cdot contor(w_2)}$$

- Intuiție: cu cât w_1 este mai *independent* de w_2, cu atât $p(w_1, w_2)$ este mai apropiată de $p(w_1) \cdot p(w_2) \longrightarrow$ cu atât mai mică este metrica *PMI*
- Dezavantaje: matrice foarte mare ($O(V^2)$); vectori foarte lungi; nu se pot compara vectori de embedding estimați pe două corpusuri diferite (decât dacă au aceeași dimensiune de vocabular)

	leash	walk	run	owner	pet	barked	the
dog	2.75	2.24	3.16	2.24	2.75	3.16	1.77
lion	0	2.75	3.16	0	3.85	0	2.06
car	0	0	3.85	2.75	0	0	2.75

Cum modelăm contextul cuvântului? **Pe bază de predicție**

- **Învățare** directă a unei reprezentări de dimensiune dată
- Utilizare a ipotezei de **distribuție a cuvintelor**
- Învățare a predicției contextului, pe baza unei **parametrizări dense**

Exemplu: modelul **word2vec skip-gram**

Model Word2Vec Skip-Gram

- Pentru fiecare document
  1. Alegem un **cuvânt țintă**
  2. Prezicem **cuvinte în context**, definite drept cuvinte aflate într-o **fereastră** de dimensiune fixă de-o parte și de alta a cuvântului țintă



Model Word2Vec Skip-Gram

- Se dau $d \in N, \mathbf{W} \in \mathbb{R}^{(V,d)}, \mathbf{C} \in \mathbb{R}^{(V,d)}$ reprezentările (embeddings) pentru cuvintele țintă și context. Pentru fiecare secvență de cuvinte din documente $(w_1, w_2, ..., w_T)$:
  - Selectăm cuvântul *focus* w, având asociat vectorul de embedding $\mathbf{w} \in \mathbb{R}^d$
  - Selectăm un cuvânt c din contextul lui w, cu vectorul de embedding $\mathbf{c} \in \mathbb{R}^d$
  - Maximizăm $\max_{\mathbf{W} \in \mathbb{R}^{(V,d)}, \mathbf{C} \in \mathbb{R}^{(V,d)}} log(p(c|w))$ – estimator **maximum likelihood**

Model Word2Vec Skip-Gram

- Cum definim $p(c|w)$?

$$p(c|w) = \frac{e^{\mathbf{w}\cdot\mathbf{c}}}{\sum_{v \in N_k} e^{\mathbf{w}\cdot\mathbf{v}}}$$

  - Targetul nostru de prezis, c, este un cuvânt din contextul lui $w \Longrightarrow$ numărul de clase posibil este dimensiunea **vocabularului** ($\longrightarrow$ problemă pentru Softmax)
  - Soluție: alegem un număr $k \ll |V|$, astfel încât N_k este un set de k cuvinte **din afara** contextului lui w

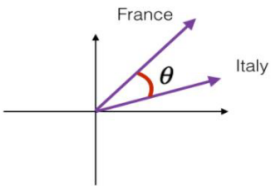
Utilizare reprezentare vectorială a cuvintelor: relații semantice exprimate aritmetic

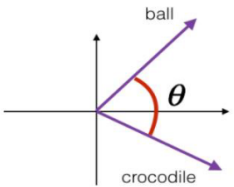
A	B	C	D=C+(B−A)	Relationship
Athens	Greece	Oslo	Norway	*Capital*
Astana	Kazakhstan	Harare	Zimbabwe	*Capital*
Angola	kwanza	Iran	rial	*Currency*
copper	Cu	gold	Au	*Atomic Symbol*
Microsoft	Windows	Google	Android	*Operating System*
New York	New York Times	Baltimore	Baltimore Sun	*Newspaper*
Berlusconi	Silvio	Obama	Barack	*First name*
Switzerland	Swiss	Cambodia	Cambodian	*Nationality*
Einstein	scientist	Picasso	painter	*Occupation*
brother	sister	grandson	granddaughter	*Family Relation*
Chicago	Illinois	Stockton	California	*State*
possibly	impossibly	ethical	unethical	*Negative*
mouse	mice	dollar	dollars	*Plural*
easy	easiest	lucky	luckiest	*Superlative*
walking	walked	swimming	swam	*Past tense*

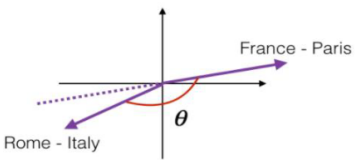
- Există dicționare preantrenate de vectori: WORD2VEC, GloVe(Global Vectors), FASTTEXT

Reprezentare numerică a textului : Word2Vec

Utilizare reprezentare vectorială a cuvintelor: similaritate între cuvinte sau expresii





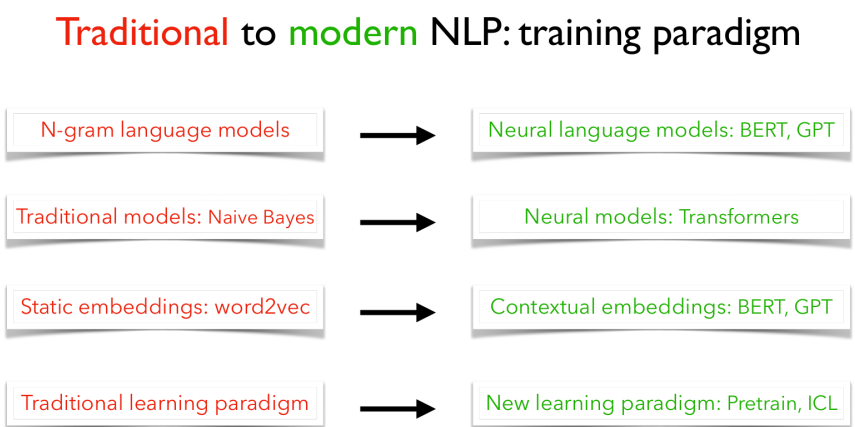


Distanța cosinus între doi vectori: $sim(v_1, v_2) = cos(\theta) = \frac{v_1 \cdot v_2}{\|v_1\| \cdot \|v_2\|}$

Modele Mari de Limbaj (Large Language Models – LLM)

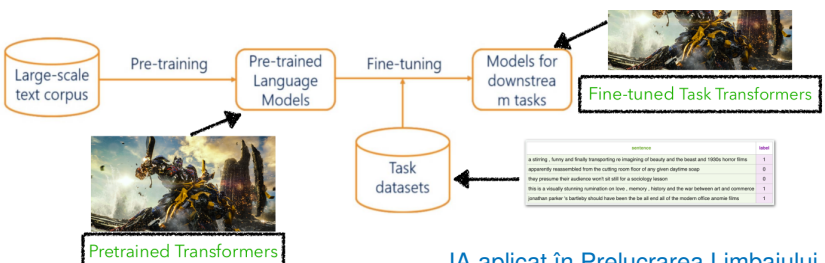
- Cum se face antrenarea lor?
- Cum se utilizează în mod inteligent capacitatea lor de generare de text?

Trecere de la abordări **"tradiționale"** la **actuale** în ML pentru NLP

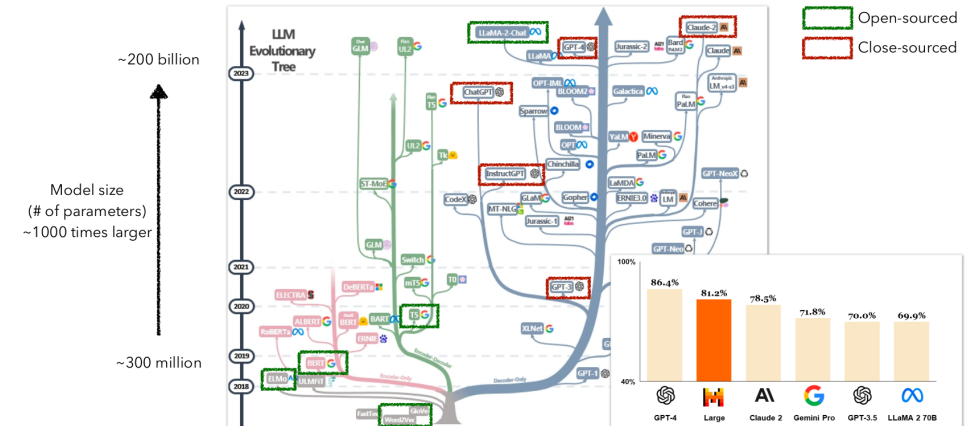


Diferență esențială în **metoda de antrenare**:

- **"Tradițional"**
  - **Învățare supervizată + finetuning**, fără pre-antrenare
  - Modelele (e.g. Regresie Logistică, Naive Bayes, RNN, LSTM) vor funcționa bine **doar** pe task-ul pentru care sunt antrenate
- **Actual**
  - Pre-antrenare + Învățare supervizată/fine-tuning
  - **Pre-antrenare**: task de antrenare, adesea **nesupervizat**, pe **foarte mult** text general
  - Modelul pre-antrenat este apoi "tunat fin" pe task-ul de predicție dorit



Evoluție LLM



https://github.com/Mooler0410/LLMsPracticalGuide
https://mistral.ai/news/mistral-large/

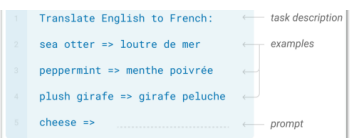
Sursa: https://github.com/Mooler0410/LLMsPracticalGuide

Învățare cu LLMs

- **Pre-antrenare** + Prompting / învățare *în context*
  - Se **pre-antrenează** un model (**foarte mare** $\longrightarrow$ 7 mld, $\approx$ 175 mld de parametrii) folosind **foarte mult** text (e.g. GPT3 $\approx$ 570 GB text = $\approx$ 400 mld tokeni).
  - Se poate **folosi direct** modelul pre-antrenat (fără fine-tuning suplimentar) pentru orice task plecând de la (i) **descrierea task-ului**, (ii) exemple în limbaj natural de perechi (intrare, ieșire). (i) și (ii) sunt numite tehnici de prompting / învățare în context.



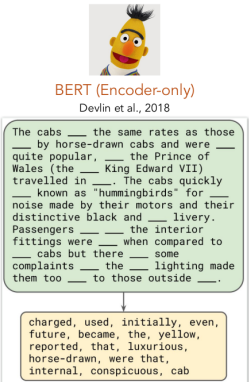
Zero-shot prompting



Few-shot prompting/in-context learning

LLM : Obiective de antrenare

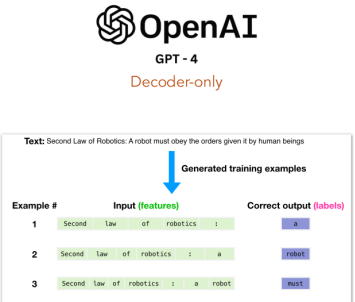
Care sunt **obiectivele de pre-antrenare** pentru un LLM? Care sunt țintele predicției?



Masked token prediction



Denoising span-mask prediction

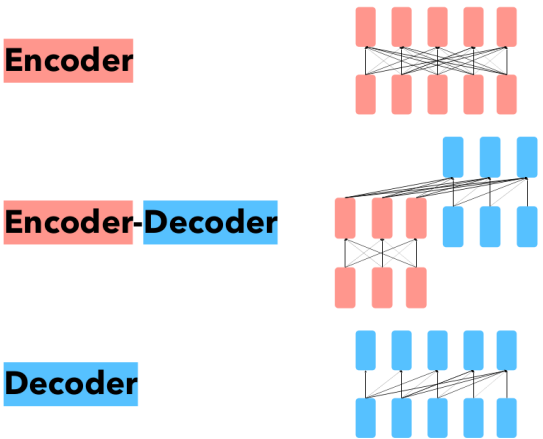


Next token prediction

Sursa: https://github.com/manueldeprada/Pretraining-T5-PyTorch-Lightning

LLM : Obiective de antrenare

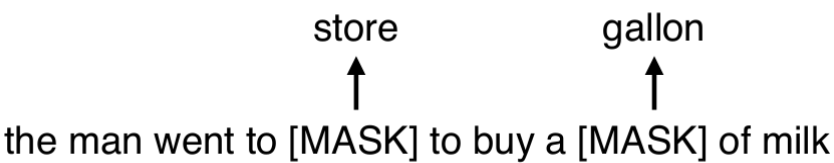
Care sunt arhitecturile tipice pentru LLM-uri?



- **Encoder-only**: exemple modele – BERT, DeBERTa, RoBERT; antrenare folosind **mascare de limbaj** (masked language modeling)
- **Encoder-decoder**: exemple modele – T5, BART; antrenare prin **predicție de mască de interval** (span-mask prediction)
- **Decoder only**: exemple modele – GPT2, GPT3, GPT4, RoLLama; antrenare prin **predicția următorului token** – $p(w_i|w_{i-1}, w_{i-2}, \ldots, w_1)$

Masked Language Modeling (MLM)

- Se maschează k% (de regulă k=15%) din cuvintele dintr-o propoziție de intrare și se **prezic** cuvintele mascate

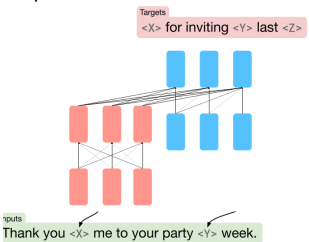


Span-mask prediction

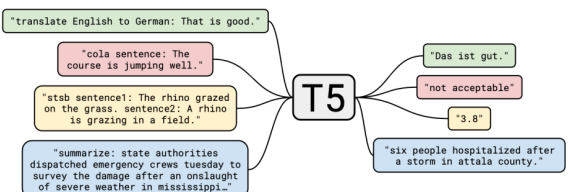
- Modelele encoder-only (antrenate prin MLM) nu pot fi utilizate pentru a *genera* text
- Modelele decoder-only pot *doar* genera text de la "stânga la dreapta"

⇒ Putem folosi modele text-to-text (encoder-decoder) pentru a fructifica ambele avantaje

- Span-mask prediction: înlocuire a diferitelor intervale de text cu măști cu id-uri
- La decodare, conținutul măștilor cu id-uri sunt prezise în maniera "de la stânga la dreapta"

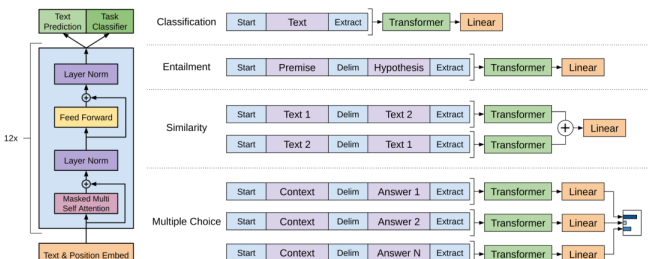


T5 = **T**ext-**t**o-**T**ext **T**ransfer **T**ransformer



Modele decoder only – antrenare **model de limbaj**

- Folosesc predicția următorului token ca obiectiv de antrenare $p(w_i|w_{i-1}, w_{i-2}, \ldots, w_1)$
- Toate LLM-urile "celebre" azi sunt decoder-only: seria GPT, LLama, Claude, Gemini
- Pot prezice condiționat de un context suplimentar $\mathbf{x}$: $p(w_i|w_{i-1}, w_{i-2}, \ldots, w_1, \mathbf{x})$
- Un model pre-antrenat poate fi ajustat (finetuned) pe task-uri specifice (aranjate ca predicție de token următor) pentru a deveni mai bun la generarea de limbaj



Sursa: GPT finetuning tasks: Redford et al., 2018

Tehnici de prompting – https://www.promptingguide.ai/

- **Claritate a cerinței** - utilizare a unei acțiuni clare așteptate (e.g. scrie, clasifică, tradu, ordonează)
- Scriere **cât mai specifică** a cerinței (a nu se lăsa liber interpretării)
- Exemple de tehnici avansate de prompting:
  - Few-shot prompting
  - Chain-of-Thought prompting
  - Prompt Chaining
  - Retrieval Augmented Generation
  - ReAct (Reason + Act)

Mulțumesc!

https://forms.gle/DJUdkejstkvyNRF5A

Feedbackul este binevenit!